

# Information with SVGD

Ethan K. Gordon

Last Updated: May 18, 2026

## Contents

|          |                                        |          |
|----------|----------------------------------------|----------|
| <b>1</b> | <b>Problem Definition and Notation</b> | <b>1</b> |
| 1.1      | Measurement Model . . . . .            | 2        |
| <b>2</b> | <b>Algorithm</b>                       | <b>2</b> |
| 2.1      | Sampling . . . . .                     | 2        |
| 2.2      | Gradient Computation . . . . .         | 3        |

## 1 Problem Definition and Notation

Assume we have an object defined by geometric parameters  $\theta$  located at state  $x_t$ . Normally, a state consists of the a position  $q_t$  and a velocity  $v_t$ , but for simplicity our state will only consist of the position. The velocity can be imputed from backwards Euler finite differences:  $v_{t+1} = q_{t+1} - q_t$ , with the assumption  $v_0 = 0$ . We also have a robot that has taken trajectory  $r_t$ . Each timestep, we have also collected measurements  $m_t$ , and we have access to a normalized per-timestep measurement model  $p_\theta(m_t|x_t)$ . We also assume that we have access to a forward/backward state transition likelihood in the exponential family:

$$\log p_\theta(x_t|x_{t+1}) \propto f_\theta(x_t, x_{t+1}) \quad (1)$$

In practice,  $f$  is the violation-implicit (vimp) loss (without any measurement terms).

Assume we have previous run a learning algorithm to compute a maximum-likelihood estimate (MLE) of our learned parameters  $\omega$ , which include the object state at the present timestep  $T$ :  $\omega = (\theta, x_T)$ . The maximum likelihood is marked with a tilde, e.g.  $\tilde{\omega}$ . During the learning process, we have also learned some maximum-likelihood trajectory  $\tilde{x}_t$ .

Our first objective is to compute the Observed Information  $\mathcal{I}$ , defined as the empirical variance of the gradient of the measurement likelihood evaluated at the MLE.

$$\mathcal{I}(\tilde{\omega}, m_t, r_t) := \frac{1}{T} \sum_t (\nabla_\omega \log p_\theta(m_t|x_T)) (\nabla_\omega \log p_\theta(m_t|x_T))^T \quad (2)$$

## 1.1 Measurement Model

Currently, each timestep there are two measurements per robot sensor  $m_t = \{c_t, n_t\}$ , the contact boolean  $c_t \in \{0, 1\}$  (i.e.  $c_t = 1$  if contact is made), and the contact normal  $n_t \in SO(2)$  (though it is represented in  $R^3$ ). We can directly write down the log measurement loss as follows:

$$\log p_\theta(m_t|x_t) = - \sum_{N_r} c_t \frac{1}{2} (\sigma_n^{-2} (1 - \hat{n}_t(\theta, x_t) \cdot n_t) + \log(2\pi\sigma_n^2)) \quad (3)$$

$$+ (c_t - 1) \alpha \phi_t(\theta, x_t) + \log(1 + e^{\alpha \phi_t(\theta, x_t)}) \quad (4)$$

Where:

- $N_r$  is the number of robot sensors
- $\phi_t(\theta, x_t)$  is the signed distance between the object and each robot sensor
- $\hat{n}_t(\theta, x_t)$  is the calculated contact normal between the object and each robot sensor
- $\alpha$  and  $\sigma_n$  are variance hyperparameters

Notes:

- The constant drops out of the (3) for the gradient, but the direct value is needed for some *softmax* computations below.
- $-\log(\text{sigmoid}(-x)) = \log(1 + \exp(x))$ . This is more numerically stable.

## 2 Algorithm

In Summary:

- Sample:** Given  $\omega = (\theta, x_T)$ , sample  $x_{t < T} \sim p_\theta(\cdot|x_T)$  by recursively sampling from the (approximate) marginal distribution  $\int p_\theta(x_t|x_{t+1})p_\theta(x_{t+1}|x_T)dx_{t+1}$ .
- Gradient:** For each timestep, use the sampled states to compute the Jacobian of the measurement likelihood given the parameters:  $\nabla_\omega \log p_\theta(m_t|x_T)$
- Return:**  $\mathcal{I}$  as defined above

### 2.1 Sampling

Draw  $N_i$  samples of each timestep's state, with each sample index by  $i$ , such that  $x_{t,i}$  is the  $i$ 'th sample at the  $t$ 'th timestep. We can do this through iterative Stein Variation Gradient Descent (SVGD), with  $s$  being the iteration number. We initialize from a Gaussian prior

with given covariance  $\Sigma_s$  about our maximum likelihood trajectory:  $x_{t,i}^0 \sim \mathcal{N}(\tilde{x}_t, \Sigma_s)$ . Then we iterate:

$$x_{t,i}^{s+1} = x_{t,i}^s + \frac{\epsilon}{N_i} \sum_j^{N_i} \left[ k(x_{t,j}^s, x_{t,i}^s) \nabla_{x_{t,j}^s} \log p_\theta(x_{t,j}^s | x_T) + \nabla_{x_{t,j}^s} k(x_{t,j}^s, x_{t,i}^s) \right] \quad (5)$$

Where  $\epsilon$  is the learning rate. The kernel  $k$  is defined as the radial basis function with the median bandwidth heuristic:

$$k(x, x') = \exp \left( -\frac{\|x - x'\|_2^2}{2h^2} \right) \quad (6)$$

Where  $h = \tilde{d}^2 / \log N_i$ , and  $\tilde{d}$  is the median of the pairwise distances between all particles. Finally, the gradient log-likelihood  $\nabla_{x_{t,i}^s} \log p_\theta(x_{t,i}^s | x_T)$  can be computed using the current samples of the following distribution, as follows:

$$\nabla_{x_{T-1}} \log p_\theta(x_{T-1} | x_T) = \nabla_{x_{T-1}} f_\theta(x_{T-1}, x_T) \quad (7)$$

$$\nabla_{x_t} \log p_\theta(x_t | x_T) \approx \nabla_{x_t} \log \sum_{x_{t+1,i} \sim p_\theta(x_{t+1} | x_T)} p_\theta(x_t | x_{t+1,i}) \quad (8)$$

$$= \nabla_{x_t} \log \sum_i \exp f_\theta(x_t, x_{t+1,i}) \quad (9)$$

$$= \text{softmax}_i(f_\theta(x_t, x_{t+1,i})) \cdot \nabla_{x_t} f_\theta(x_t, x_{t+1,i}) \quad (10)$$

Using the fact that  $\nabla \log \sum \exp = \text{softmax}$ , and the dot product happening over the  $i$  dimension.

## 2.2 Gradient Computation

Given per-timestep samples  $x_{t,i}$ , compute the Jacobian-measurement-log-likelihood  $\nabla_\omega \log p_\theta(m_t | x_T)$ . Importantly, we should ideally not take the gradient through the above sampling procedure (which can be unstable). Previous work has shown that the gradient can be written as follows:

$$\nabla_\omega \log p_\theta(m_t | x_T) \quad (11)$$

$$= \text{softmax}_i[\log p_\theta(m_t | x_{t,i})] \cdot \nabla_\omega (\log p_\theta(m_t | x_{t,i}) + \log p_\theta(x_{t,i} | x_T)) - \frac{1}{N_i} \sum_i \nabla_\omega (\log p_\theta(x_{t,i} | x_T)) \quad (12)$$

With the dot product happening over the  $i$  dimension.

Using this, we can also recursively define the state gradient-log-likelihood  $\nabla_\omega \log p_\theta(x_{t,i} | x_T)$

$$\nabla_\omega \log p_\theta(x_T | x_T) = 0 \quad (13)$$

$$\nabla_{\omega} \log p_{\theta}(x_{t,i}|x_T) \tag{14}$$

$$= \textit{softmax}_j[\log p_{\theta}(x_{t,i}|x_{t+1,j})] \cdot \nabla_{\omega}(\log p_{\theta}(x_{t,i}|x_{t+1,j}) + \log p_{\theta}(x_{t+1,j}|x_T)) - \frac{1}{N_j} \sum_j \nabla_{\omega}(\log p_{\theta}(x_{t+1,j}|x_T)) \tag{15}$$

$$= \textit{softmax}_j[f_{\theta}(x_{t,i}, x_{t+1,j})] \cdot \nabla_{\omega}(f_{\theta}(x_{t,i}, x_{t+1,j}) + \log p_{\theta}(x_{t+1,j}|x_T)) - \frac{1}{N_j} \sum_j \nabla_{\omega}(\log p_{\theta}(x_{t+1,j}|x_T)) \tag{16}$$

With the dot product happening over the  $j$  dimension.